Name _____

CSCI 332, Fall 2026

Exam 1 (Alternate Version)

This exam consists of the following sections:

1. Stable Matching (25 points)
2. Algorithm Analysis (25 points)
3. Recursion (25 points)
4. 1D Dynamic Programming (25 points)

You may use a double-sided 3×5 handwritten notecard of notes during the test but no other resources.

Good luck!

Section 1 (Stable Matching)

1. (15 points total) Consider the preference lists below, listed from most preferred to least preferred, and the proposed matching

$$M = \{(h_1, s_2), (h_2, s_4), (h_3, s_1), (h_4, s_3)\}.$$

Hospital	1st	2nd	3rd	4th
h_1	s_4	s_2	s_1	s_3
h_2	s_2	s_1	s_4	s_3
h_3	s_3	s_1	s_4	s_2
h_4	s_1	s_3	s_2	s_4
Student	1st	2nd	3rd	4th
s_1	h_3	h_2	h_4	h_1
s_2	h_2	h_1	h_4	h_3
s_3	h_4	h_3	h_1	h_2
s_4	h_1	h_2	h_3	h_4

For each pair below, circle whether it is an **unstable pair with respect to** M and briefly explain why. Each part is worth 5 points.

(a) (h_1, s_4) Yes No

(b) (h_2, s_1) Yes No

(c) (h_4, s_3) Yes No

2. (10 points) In the **stable roommate problem**, there is one set of an even number of people. Each person ranks every other person in strict order of preference. A matching divides everyone into disjoint pairs, so that every person has exactly one roommate.

   An **unstable pair** is a pair of people who are not matched to each other and who each prefer the other to their assigned roommate. A matching is **stable** if it has no unstable pair.

   Consider the following instance. Preferences are listed from most preferred to least preferred.

Person	1st	2nd	3rd
Alex	Blair	Casey	Drew
Blair	Casey	Alex	Drew
Casey	Alex	Blair	Drew
Drew	Alex	Blair	Casey

   Does a stable roommate matching exist for this instance? Justify your answer. If you claim that none exists, explain why **every possible matching** has an unstable pair.

Section 2 (Algorithm Analysis)

3. (9 points total) For each exact runtime $T(n)$ below, compute and simplify the **doubling ratio** $T(2n)/T(n)$. These are exact runtimes, not asymptotic bounds. Assume $n > 1$. Each part is worth 3 points.

   (a) $T(n) = 7n^3$

   (b) $T(n) = 5\log_3 n$

   (c) $T(n) = 4 \cdot 2^n$

4. (5 points) Recall: $f(n)$ is $\Omega(g(n))$ if there exist positive constants c and n_0 such that $f(n) \geq c \cdot g(n)$ for every integer $n \geq n_0$.

   Select all pairs (n_0, c) below that can be used to show that $6n^2 + 3n$ is $\Omega(n^2)$.

   A. $n_0 = 1$, $c = 6$

   B. $n_0 = 1$, $c = 9$

   C. $n_0 = 3$, $c = 5$

   D. $n_0 = 10$, $c = 7$

   E. $n_0 = 2$, $c = \frac{1}{2}$

5. (6 points) Order the functions from asymptotically smallest to asymptotically largest. Label the smallest growth class 1, the next distinct growth class 2, and so on. Give the same number to functions that are Θ of each other.

___ $n^{3/2}$

___ $\log_5(n^5)$

___ $n \log_2 n$

___ $2^{\log_2 n}$

___ $(\log_{10} n)^2$

___ $11 \log_2 n + 7$

6. (5 points) Circle all functions $f(n)$ such that the runtime of Algorithm-1 below is $\Theta(f(n))$.

```
Algorithm-1(array A[1..n]):
    Result = 0
    For i from 1 to n:
        j = 1
        While j ≤ n:
            Result = Result + A[j]
            j = 2j
    Return Result
```

A. n

B. $\log n$

C. $n \log n$

D. n^2

E. n^3

Section 3 (Recursion)

7. (15 points total) Consider $T(n) = 3T(n/3) + n$, with $T(1) = 1$.

   (a) (5 points) Draw the first three levels of the recursion tree in the left boxes, labeling each node with its nonrecursive work. Write the total work at each level in the right boxes.

Level 0	Total work
Level 1	Total work
Level 2	Total work

   (b) (3 points) Give the total work at level ℓ in terms of n and ℓ.

   (c) (2 points) Is the work per level increasing, decreasing, or the same? Circle one.

   (d) (3 points) How many levels are in the tree, as a function of n?

   (e) (2 points) Give a tight asymptotic bound for $T(n)$ using Θ notation.

8. (10 points) Given the following (meaningless) divide and conquer algorithm, what is the recurrence relation that describes its running time?

```
wow(integer array A of length n):
  if n ≤ 1:
      return A
  else:
      Let A_1,A_2,A_3 be the first third, middle third, and last third of A
      X_1 = wow(A_1)
      X_2 = wow(A_2)
      X_3 = wow(A_3)
      let array C be a new array of length n
      for i from 1 to n:
          for j from 1 to n:
              C[i%j] = first element of X_1 + middle element of X_2 + last element of X_3
      return C
```

$T(1) =$ _____

$T(n) =$ _____ + _____

work done by recursive calls | nonrecursive work

Section 4 (1D Dynamic Programming)

9. (8 points total) A set of monkey bars has bars numbered 0 through n. You start hanging from bar 0 and want to reach bar n. On each swing, you can move forward either 1 bar or 2 bars. Two ways are different if their sequences of swing lengths differ.

   Let $Ways(i)$ be the number of ways to reach bar i from bar 0 using these swings.

   (a) (2 points) What is $Ways(4)$?

   (b) (6 points) Write the recursive definition for $Ways(i)$ for all integers $i \geq 0$, including the necessary base cases. You may not need every line provided.

$$Ways(i) = \begin{cases} _____ & \text{if } i _____ \\ _____ & \text{if } i _____ \\ _____ & \text{if } i _____ \\ _____ & \text{if } i _____ \\ _____ & \text{if } i _____ \end{cases}$$

10. (9 points total) Consider the following meaningless recursive definition, where $H[1..n]$ is a given array:

$$wut(i) = \begin{cases} 2 & \text{if } i = 1, \\ H[2] & \text{if } i = 2, \\ -1 & \text{if } i = 3, \\ \max\{H[i] + wut(i-1),\ 2wut(i-3)\} & \text{if } 4 \le i \le n. \end{cases}$$

(a) (2 points) We will store $wut(i)$ in $M[i]$. Should we fill the array from left to right or right to left?

(b) (7 points) Complete the pseudocode to compute $wut(n)$, given $H[1..n]$. Include the base cases, the loop that fills the table, and the return statement.

```
MemoizedWut(H[1..n]):
  Let M[1..n] be a new array
  # initialize the base cases




  # fill the rest of the array




  # return the final answer
```

11. (8 points total) Consider the following variant of the text segmentation problem from Homework 6: Given an array $A[1..n]$ of characters, compute the number of unique partitions of A into words. For example, for `ARTISTOIL`, the two partitions are `ARTIST OIL` and `ART IS TOIL` (assuming these are the only valid partitions).

Assume you have a subroutine IsWord that takes an array of characters as input and returns True if and only if that string is a word. Words are nonempty, and two partitions are different if their boundary positions differ.

Recall the following recursive algorithm for deciding whether the suffix $A[i..n]$ can be partitioned into words. The array A and its length n are available to every recursive call.

```
Splittable(i):
  if i > n:
      return True
  for j from i to n:
      if IsWord(A[i..j]):
          if Splittable(j + 1):
              return True
  return False
```

Now let $CountSplits(i)$ be the **number of unique partitions** of the suffix $A[i..n]$ into words.

(a) (2 points) If $A =$ `aaaa` and the only words that IsWord recognizes are `a` and `aa`, what is $CountSplits(1)$?

(b) (6 points) Write a **recursive algorithm** $CountSplits(i)$ to compute this count, using the algorithm above as a starting point. Include the base case for the empty suffix. Your algorithm should return 0 if there is no valid partition. The final answer is $CountSplits(1)$. You do not need to memoize your algorithm.